

Manual del Desarrollador

Proyecto Node-RED — Seguridad en Casa

Documento técnico de referencia para mantenimiento, extensión y despliegue del sistema de seguridad.

Desarrollado por:

Debbie Milena Aguilar Lima

Melissa Yamileth Garrido López

Contenido

1. Descripción General del Proyecto.....	3
2. Requisitos del Sistema	4
2.1 Software base	4
2.2 Instalación de librerías	4
3. Arquitectura del Sistema.....	5
3.1 Vista general	5
3.2 Flujo de datos.....	5
4. Catálogo de Nodos.....	6
4.1 Resumen por tipo	6
4.2 Nodos de interfaz de usuario (ui-button)	7
4.3 Nodos de sensores (ui-switch)	8
4.4 Nodos de transformación (change)	9
4.5 Templates de visualización (ui-template)	10
4.6 Integración del Gemelo Digital 3D (ui-template)	10
5. Lógica de Negocio	11
5.1 Estado global — homeState.....	11
5.2 Cálculo del índice de riesgo.....	11
5.3 Grupos funcionales del editor	12
6. Integración con Telegram	13
7. Guía de Despliegue	14
8. Mantenimiento	14
8.1 Tabla de IDs de nodos críticos.....	14

1. Descripción General del Proyecto

El proyecto Seguridad en Casa constituye una solución integral orientada al monitoreo y la gestión inteligente de la seguridad residencial. Desarrollado íntegramente sobre la plataforma Node-RED, el sistema actúa como un nodo central de control capaz de procesar eventos de dispositivos de entrada (puertas, ventanas y sensores PIR) y gestionar estados de alarma de forma automatizada.

El sistema se caracteriza por una arquitectura basada en el flujo de estados globales, donde toda la lógica de control reside en la variable persistente `homeState`. Esto permite que la infraestructura sea escalable y altamente reactiva, logrando que cualquier cambio en la entrada del sistema se propague instantáneamente a los módulos de visualización y notificación.

La solución se estructura bajo los siguientes pilares tecnológicos:

- **Procesamiento de Eventos:** El flujo, compuesto por 48 nodos organizados en grupos funcionales, utiliza lógica de programación visual y sentencias JSONata para la manipulación de datos en tiempo real.
- **Gestión de Riesgos Dinámica:** El sistema no solo monitorea el estado binario (abierto/cerrado o movimiento detectado), sino que calcula de forma dinámica un Índice de Riesgo (0–100). Este cálculo se basa en un algoritmo de pesos ponderados que evalúa la criticidad de cada evento, proporcionando una métrica cuantificable de la vulnerabilidad de la vivienda en cualquier instante.
- **Interfaz y Notificación:** La experiencia de usuario se centraliza en un dashboard web altamente responsivo y personalizado que integra elementos de visualización 2D y 3D. Adicionalmente, el sistema se extiende hacia la comunicación externa mediante la integración de notificaciones push vía Telegram, lo que garantiza que el usuario reciba alertas críticas en tiempo real independientemente de su ubicación geográfica, cerrando así el ciclo de seguridad desde el hardware hasta el usuario final.

2. Requisitos del Sistema

2.1 Software base

Tabla 1: Dependencias de software

Componente	Descripción	Versión mínima
Node.js	Entorno de ejecución JavaScript	18.x LTS
npm	Gestor de paquetes Node	9.x
Node-RED	Plataforma de flujos visuales	3.1.x
@flowfuse/node-red-dashboard	Dashboard 2 (nodos ui-*)	1.x
node-red-contrib-telegrambot	Nodo Telegram Bot	14.x

2.2 Instalación de librerías

```
# Desde el directorio de Node-RED (~/.node-red)

npm install @flowfuse/node-red-dashboard

npm install node-red-contrib-telegrambot

# Reiniciar Node-RED

node-red-restart      # systemd

# o

pm2 restart node-red   # pm2
```

Tabla 2: Librerías Node-RED utilizadas en el proyecto

Librería / Paquete	Tipo de nodo	Función en el proyecto
@flowfuse/node-red-dashboard	ui-button, ui-switch, ui-template, ui-page, ui-group, ui-base, ui-theme	Interfaz visual dashboard en navegador
node-red-contrib-telegrambot	telegram bot, telegram sender	Notificaciones y alertas por Telegram
Node-RED core	change, function, delay, debug	Lógica de negocio, transformación y retardo

3. Arquitectura del Sistema

3.1 Vista general

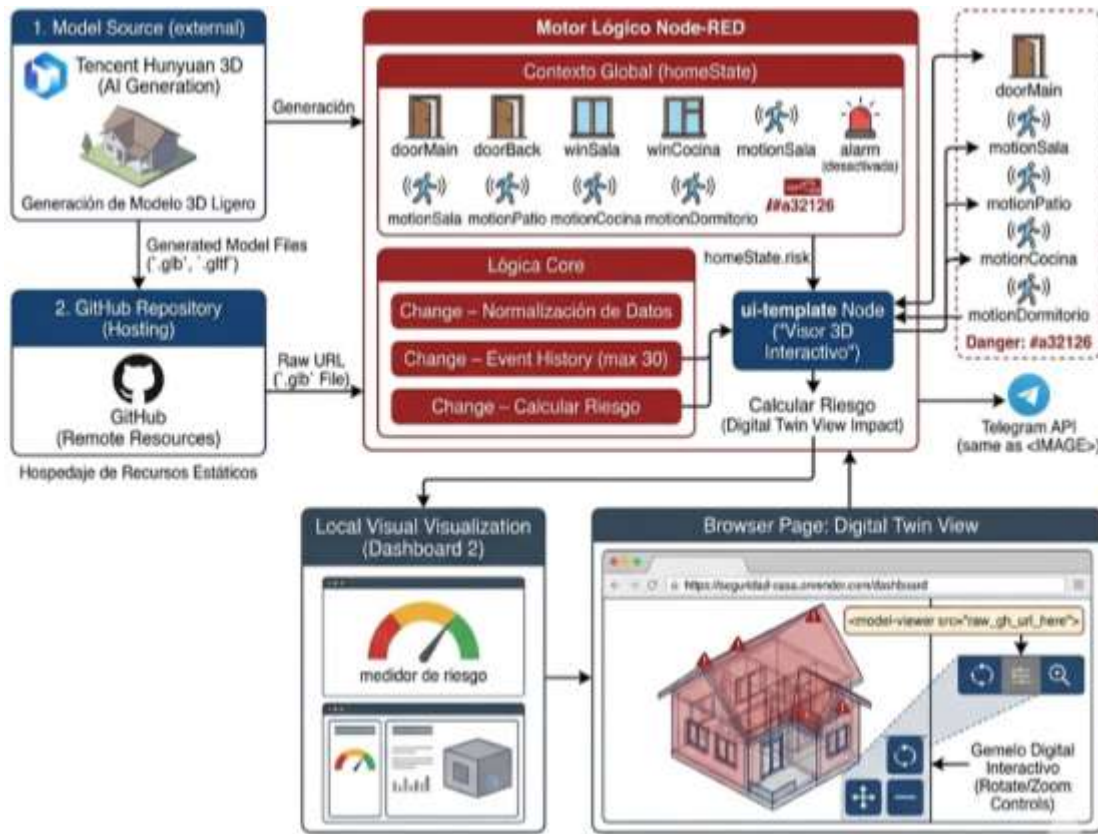


Figura 1: Diagrama de arquitectura — Seguridad en Casa

3.2 Flujo de datos

1. El usuario interactúa con botones/switches en el dashboard.
2. Cada nodo UI emite un mensaje con topic (identificador del dispositivo) y payload (estado).
3. Un nodo Change inicializa homeState si no existe, y actualiza el campo correspondiente en el contexto global.
4. El nodo Change – Generar Historial añade una entrada timestamped al arreglo homeState.history (máx. 30 entradas).
5. El nodo Change – Calcular Riesgo calcula el índice numérico y lo almacena en homeState.risk.
6. Cuatro nodos ui-template reciben el estado completo para actualizar el dashboard (medidor, tabla, gemelo visual, visor 3D).
7. El nodo Function – Formatear Telegram construye el mensaje de alerta y lo envía con Telegram Sender (con throttle de 10 segundos).

4. Catálogo de Nodos

4.1 Resumen por tipo

Tabla 3: Inventario de nodos por tipo

Tipo de nodo	Cantidad	Función principal	Librería
tab	1	Contenedor principal del flow	Core
group	3	Agrupación visual en el editor	Core
ui-base	1	Configuración base del dashboard	Dashboard 2
ui-theme	1	Tema visual (Neon Dashboard Theme)	Dashboard 2
ui_base	1	Base legacy (compatibilidad)	Dashboard 2
ui_tab	1	Tab dashboard (CASA INTELIGENTE)	Dashboard 2
ui-page	1	Página: Dashboard Seguridad en Casa	Dashboard 2
ui-group	6	Paneles de agrupación en la UI	Dashboard 2
ui-button	10	Botones de acción del usuario	Dashboard 2
ui-switch	3	Switches de sensores PIR	Dashboard 2
ui-template	4	Widgets HTML/JS personalizados	Dashboard 2
telegram bot	1	Configuración del bot de Telegram	Telegrambot
telegram sender	1	Envío de mensajes a Telegram	Telegrambot
change	10	Modificación del contexto global y payloads	Core
function	1	Formateo de mensaje de alerta	Core
delay	1	Throttle de 10 s para Telegram	Core
debug	1	Depuración de salida Telegram	Core
global-config	1	Configuración global del flow	Core
Total	48		

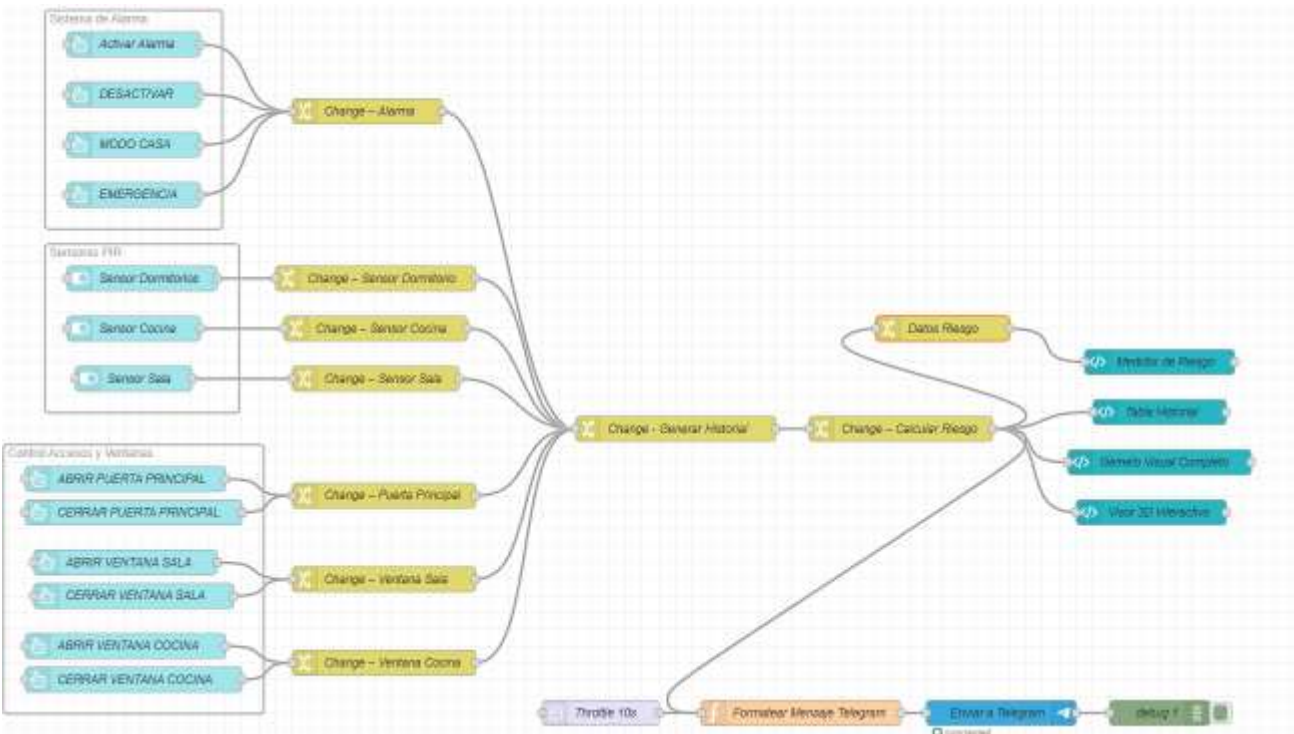


Figura 1: Diagrama de arquitectura — Seguridad en Casa

4.2 Nodos de interfaz de usuario (ui-button)

Tabla 4: Botones del dashboard

Nombre	Topic	Payload	Grupo UI
Activar Alarma	alarm	activada	Sistema de Alarma
Desactivar	alarm	desactivada	Sistema de Alarma
Modo Casa	alarm	casa	Sistema de Alarma
Emergencia	alarm	emergencia	Sistema de Alarma
Abrir Puerta Principal	doorMain	true	Control Accesos
Cerrar Puerta Principal	doorMain	false	Control Accesos
Abrir Ventana Sala	winSala	true	Control Accesos
Cerrar Ventana Sala	winSala	false	Control Accesos
Abrir Ventana Cocina	winCocina	true	Control Accesos
Cerrar Ventana Cocina	winCocina	false	Control Accesos

4.3 Nodos de sensores (ui-switch)

Tabla 5: Switches PIR del dashboard

Nombre	Topic	Grupo UI	Descripción
Sensor Dormitorios	motionDormitorio	Sensores PIR	Detecta movimiento en dormitorios
Sensor Cocina	motionCocina	Sensores PIR	Detecta movimiento en cocina
Sensor Sala	motionSala	Sensores PIR	Detecta movimiento en sala



Figura 2: Configuración del nodo switch para el Sensor de Dormitorios.



Figura 3: Configuración del nodo switch para el Sensor de Cocina.

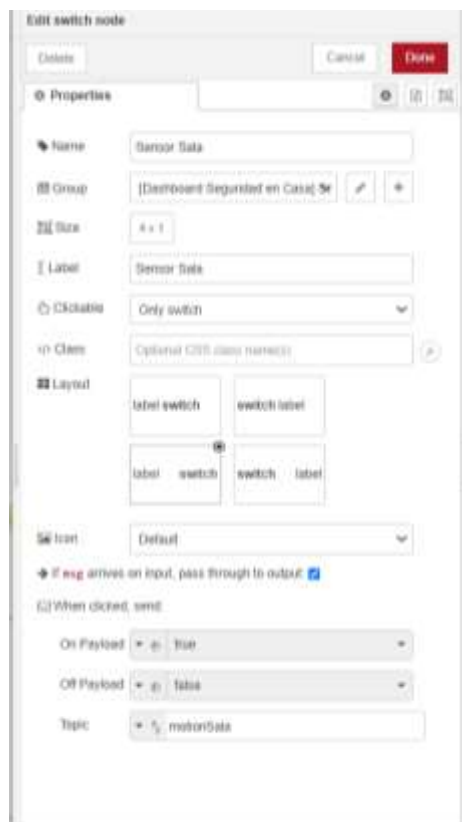


Figura 4: Configuración del nodo switch para el Sensor de Sala.

4.4 Nodos de transformación (change)

Tabla 6: Nodos Change — lógica de actualización de estado

Nombre del nodo	Campo actualizado	Destino siguiente
Change – Alarma	homeState.alarm	Change – Generar Historial
Change – Sensor Dormitorio	homeState.motionDormitorio	Change – Generar Historial
Change – Sensor Cocina	homeState.motionCocina	Change – Generar Historial
Change – Sensor Sala	homeState.motionSala	Change – Generar Historial
Change – Puerta Principal	homeState.doorMain	Change – Generar Historial
Change – Ventana Sala	homeState.winSala	Change – Generar Historial
Change – Ventana Cocina	homeState.winCocina	Change – Generar Historial
Change – Generar Historial	homeState.history (máx 30)	Change – Calcular Riesgo
Change – Calcular Riesgo	homeState.risk (0–100)	ui-template × 4 + Telegram

4.5 Templates de visualización (ui-template)

Tabla 7: Widgets personalizados del dashboard

Nombre	Grupo UI	Función
Medidor de Riesgo	Estado General (Riesgo)	Gauge visual del índice 0–100
Tabla Historial	Estado General (Riesgo)	Últimos 30 eventos timestamped
Gemelo Visual Completo	Visualización Gemelo Digital	Vista 2D del plano de la casa
Visor 3D Interactivo	Modelo 3D Interactivo	Representación 3D navegable

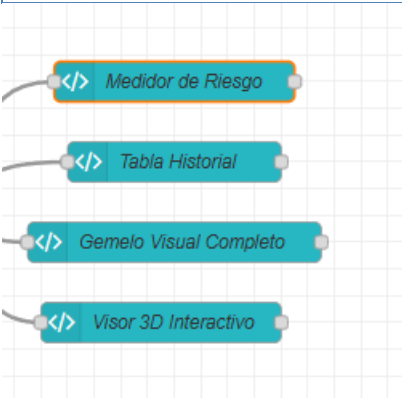


Figura 5: Nodos template personalizados.

4.6 Integración del Gemelo Digital 3D (ui-template)

El proyecto incluye una representación visual avanzada del hogar mediante un modelo 3D interactivo, funcionando como un gemelo digital ligero.

- **Generación del Modelo:** La estructura 3D fue generada utilizando la plataforma de inteligencia artificial Tencent Hunyuan 3D (<https://3d.hunyuan.tencent.com/>).
- **Alojamiento (Hosting):** Los archivos resultantes del modelo (ej. formato .glb o .gltf) están versionados y alojados en un repositorio de GitHub. Esto permite servir los recursos gráficos de manera remota, estática y sin sobrecargar el servidor local de Node-RED.
- **Código del Template:** La visualización se renderiza en el dashboard a través del nodo ui-template denominado "Visor 3D Interactivo". El código HTML y JavaScript inyectado en este nodo consume la URL en formato raw del modelo desde GitHub (utilizando componentes web como <model-viewer> o librerías gráficas estándar). Esto incrusta el visor en la interfaz, permitiendo al usuario interactuar libremente (rotar, acercar y alejar) con el plano de la casa directamente desde el navegador.

5. Lógica de Negocio

5.1 Estado global — homeState

Todos los nodos comparten estado mediante el contexto global de Node-RED. La estructura del objeto es:

```
{
  "doorMain":    false, // Puerta principal (true = abierta)
  "doorBack":    false, // Puerta trasera
  "winSala":     false, // Ventana sala
  "winCocina":   false, // Ventana cocina
  "motionSala":  false, // Movimiento sala
  "motionPatio": false, // Movimiento patio
  "motionCocina": false, // Movimiento cocina
  "motionDormitorio": false, // Movimiento dormitorio
  "alarm":       "desactivada", // activada | desactivada | casa | emergencia
  "history":     [], // Array de eventos (max 30)
  "risk":        0 // Indice de riesgo 0-100
}
```

5.2 Cálculo del índice de riesgo

El nodo Change – Calcular Riesgo evalúa cada campo con una ponderación fija. El resultado se limita a 100 con $\$min([..., 100])$.

Tabla 8: Ponderación del índice de riesgo

Condición	Puntos	Campo
Puerta principal abierta	+20	doorMain
Puerta trasera abierta	+20	doorBack
Ventana sala abierta	+10	winSala
Ventana cocina abierta	+10	winCocina
Movimiento en sala	+15	motionSala
Movimiento en patio	+15	motionPatio
Movimiento en cocina	+15	motionCocina

Movimiento en dormitorio	+15	motionDormitorio
Alarma activada	+10	alarm = activada
Emergencia	+100 (fuerza máx.)	alarm = emergencia
Máximo posible	100	risk

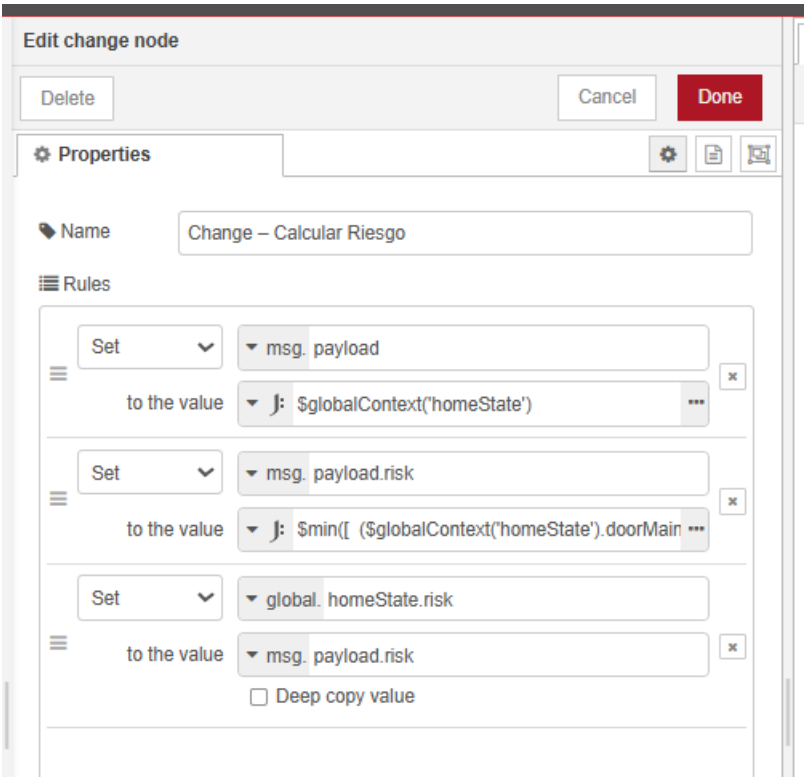


Figura 6: Configuración de payloads y topic del nodo change Clcular Riesgo.

5.3 Grupos funcionales del editor

Tabla 9: Grupos del flow (editor Node-RED)

Nombre del grupo	Nodos incluidos	Nodos
Sistema de Alarma	4 botones de alarma + Change Alarma	4
Sensores PIR	3 switches PIR + 3 nodos Change sensores	3
Control Accesos y Ventanas	4 pares open/close (puerta + 2 ventanas) + 6+ Changes	6+

6. Integración con Telegram

Tabla 10: Nodos del pipeline de Telegram

Nodo	Tipo	Función	Config ID
Telegram Bot (config)	telegram bot	Credenciales del bot (token)	telegram_config_01
Throttle 10s	delay	Limita a 1 mensaje cada 10 s	telegram_throttle_01
Formatear Mensaje	function	Construye el texto con emojis	telegram_format_01
Enviar a Telegram	telegram sender	POST a la API de Telegram	telegram_sender_01
debug 1	debug	Verifica la respuesta de la API	f0185b0113f9042d

Niveles de alerta en el mensaje

Tabla 11: Clasificación de riesgo para mensajes Telegram

Rango	Emoji	Texto	Acción recomendada
0–39	🟢	BAJO	Monitoreo pasivo
40–69	🟡	MEDIO	Verificar sensores activos
70–100	🔴	ALTO	Revisar de inmediato / notificar

Variables de entorno requeridas (definir en settings.js o en el nodo global-config):

TELEGRAM_TOKEN=<token-del-bot-obtenido-en-BotFather>

TELEGRAM_CHAT_ID=<id-del-chat-o-grupo-destino>



Figura 7: Configuración de Nodo function (Formatear Mensaje).



Figura 8: Propiedades y configuración del nodo Telegram Sender.

7. Guía de Despliegue

1. Instalar Node.js 18 LTS y Node-RED (npm i -g node-red).
2. Instalar las librerías indicadas en la Tabla 1.
3. Importar el archivo flows.json desde el menú Import de Node-RED.
4. Configurar el nodo Telegram Bot con el token del bot.
5. Hacer clic en Deploy (modo Full).
6. Acceder al dashboard en <http://<host>:1880/dashboard>.

8. Mantenimiento

8.1 Tabla de IDs de nodos críticos

Tabla 12: IDs de nodos clave para mantenimiento

Nombre	Función	ID
Change – Generar Historial	Punto de entrada unificado de todos los cambios	3733a0a8daa5eb3f
Change – Calcular Riesgo	Calcula homeState.risk y distribuye	5d19b0793c281044
Telegram Bot (config)	Credenciales del bot	telegram_config_01
Throttle 10s	Evita spam de notificaciones	telegram_throttle_01
Medidor de Riesgo	Widget de gauge en dashboard	84a1335b91305b5c
Tabla Historial	Widget de historial en dashboard	4c1d1139e6f28900
Datos Riesgo	Inyecta timestamp en milisegundos al payload del riesgo	10969d428b30553f